

OVERNIGHT QUANTITATIVE RESEARCH AUDIT · INDEPENDENT

The ProSignal Audit

An adversarial reconstruction of a 36,000-line NSE equity signal engine, from the raw bhavcopy to the shortlist — run against nine years of its own point-in-time store, with the production model reproduced coefficient for coefficient.

VERDICT · NOT YET

REPOSITORY
rahulreddyallu/prosignal @ a359622

STORE
2,212 sessions · 3,545 symbols · 2017-09
→ 2026-08

SCOPE
36,085 src + 19,234 test lines, 162
commits

REPRODUCTION
Production model matches λ , t, n_train,
n_dates exactly

Contents

01 Executive verdict

02 What ProSignal is

03 How this was audited

04 Layer scorecard

05 P0 · Validity threats

06 False-confidence register

07 The economic question

08 Ablation & evidence ledger

09 Factor registry

10 Research weaknesses

11 Testing the tests

12 Missing capabilities

13 What is genuinely strong

14 Hypotheses I rejected

15 Roadmap

16 Would I trust it?

WOULD AN INDEPENDENT RESEARCHER TRUST THIS SYSTEM'S OUTPUTS?

Not yet — and not for the reason the repository expects.

The engineering is unusually disciplined and the leakage controls the project worries about most are, with two exceptions, real. I reproduced the live model exactly — all seven λ , all seven t , $n_{\text{train}} = 43,555$, $n_{\text{dates}} = 83$ — from the store, on the first attempt. That is rare and it means everything below is about the shipped system, not an approximation of it.

What fails is not the plumbing. It is that **the engine has never once compared its book to buying the market**, and when you do, the answer is decisive: over its own selection period the book returns **+3.4% CAGR at Sharpe +0.29** against **+23.5%** for the equal-weight eligible universe and **+16.2% at Sharpe +0.84** for the Nifty200 Momentum 30 — a listed, buyable index expressing the same thesis for about thirty basis points a year. At the shipped 63-session rebalance cadence the book returns **−0.8%**.

Alongside that, the statistic the project relies on to certify itself against selection bias — the Deflated Sharpe Ratio — is arithmetically incapable of failing. It passes at 100,000 trials.

What ProSignal actually is

Stripped of its seven-theme presentation, the live engine is a two-factor cross-sectional ranker: twelve-month-ish price momentum and the NSE-specific delivered-volume share, applied long-only to eight names with a 2.5×ATR stop.

Seven families are built. Three carry a non-zero coefficient. Of those three, `mom_f` and `delivery_f` are essentially orthogonal within date ($\rho = -0.028$) and account for the model's entire measured information; the third, `lottery_f`, survives only because of a standard-error choice the repository itself argues against elsewhere in its own codebase (see F3).

THE LIVE MODEL, REPRODUCED FROM THE STORE · FITTED_FOR 2026-08-25						
FAMILY	A	T (NW-2, SHIPPED)	T (ANALYTIC VIF)	TRADED?	MEMBERS	
<code>mom_f</code>	+0.0764	+4.12	+3.27	PRICED	mom_6_1, prox_52w, resid_mom	
<code>delivery_f</code>	+0.0474	+4.77	+3.34	PRICED	deliv_pct, deliv_trend	
<code>lottery_f</code>	-0.0455	-2.26	-1.72	PRICED — SHOULD NOT BE	max5_21, idio_vol, downside_vol	
<code>reversal_f</code>	+0.0182	+1.69	+1.01	ZEROED	resid_reversal	
<code>skew_f</code>	-0.0190	-1.84	-1.46	ZEROED	idio_skew	
<code>beta_f</code>	+0.0087	+0.52	+0.43	ZEROED	beta_120	
<code>drawdown_f</code>	+0.0090	+0.51	+0.40	ZEROED	max_dd_120	

Value and quality — eleven more factors — are computed on every run and dropped for coverage on every run (date_span 0.3765 against a 0.60 floor). They have never contributed to a score.

THE GAP BETWEEN THE DESCRIBED SYSTEM AND THE SHIPPED ONE

The README is a remarkable document — self-critical to an unusual degree — but it describes a system that no longer exists. The most recent commit (a359622, "Stop fitting the ranker on the engine's own exit geometry") reverted the training label from a triple-barrier construction back to a plain 63-session forward return. The README's entire section titled *"The label is the trade, not the horizon return"* describes the removed design. The live model blob confirms it: `"label": {"triple_barrier": false, "source": "forward_return"}`.

That reversion is defensible — the commit's reasoning is correct, the barrier label's magnitude was mechanically the name's volatility — but it left the documentation, the family table, the coefficient table, the CPCV figures, the trial count, the PBO figure and the forward-test registration all describing a different engine.

How this was audited

Everything below is either a line of code I can point at, or a number I computed by running the engine's own functions against its own nine-year store. Nothing is inferred from the documentation.

I cloned the repository, installed it, copied the live curated store (218 MB of point-in-time parquet: prices, indices, delivery, statements, corporate actions, ledger), and rebuilt the production training panel with `build_panel` under the shipped config. The reproduction check came out exact:

REPRO The production model reproduces bit-for-bit from the store

CONFIRMED

theme	lambda	se(NW2)	t	live blob
mom_f	+0.0764	0.0186	+4.12	lam +0.0764 t +4.12
reversal_f	+0.0182	0.0108	+1.69	lam +0.0182 t +1.69
lottery_f	-0.0455	0.0202	-2.26	lam -0.0455 t -2.26
skew_f	-0.0190	0.0103	-1.84	lam -0.0190 t -1.84
beta_f	+0.0087	0.0168	+0.52	lam +0.0087 t +0.52
drawdown_f	+0.0090	0.0177	+0.51	lam +0.0090 t +0.51
delivery_f	+0.0474	0.0099	+4.77	lam +0.0474 t +4.77
n_train 43,555 (blob 43,555) n_dates 83 (blob 83)				

Why this matters. Every experiment that follows uses the same panel, the same estimator and the same coefficients the engine trades. Where I disagree with the repository, the disagreement is about the same numbers, not about different ones.

WHAT WAS RUN

- Full pipeline reproduction; live scoring path traced end to end against the store.
- The engine's own CPCV (45 splits, 9 paths), portfolio simulator, cost model, Fama–MacBeth estimator, significance module and decay monitor.
- A second, independently written continuous-book simulator, to cross-check the repository's own.
- Null and placebo controls: five random rankers, inverted momentum, inverted full model.
- Ablations across every single-theme and drop-one configuration, gated and ungated.
- Factor attribution against the engine's own long-short factor definitions.
- The full test suite (1,343 passed / 1 failed / 49 skipped) plus a 16-mutation probe.
- The recorded run ledger (1,679 rows, 69 dates in 2026) and the outcome ledger (834 trades).

Layer scorecard

Four kinds of correctness are graded separately, because a component can be flawless on the first three and still fail the fourth.

SOFTWARE · MATHEMATICAL · RESEARCH · ECONOMIC					
LAYER	SOFTWARE	MATHS	RESEARCH	ECONOMIC	READING
Data & store	STRONG	STRONG	WEAK	N/A	Adjustment, locking, incremental ingest and schema validation are excellent. The universe price floor is applied to back-adjusted prices, which excludes 165 names on dates they were tradeable (F4).
Features	WEAK	STRONG	ADEQUATE	UNKNOWN	Definitions match their citations closely and the no-forward-fill discipline is right. But the live scoring path reads features four sessions stale, always (F2).
Factors	STRONG	STRONG	ADEQUATE	WEAK	Two families carry real information. Five carry none. The seven-theme presentation overstates the diversification by a factor of three.
Estimator	STRONG	WEAK	WEAK	WEAK	Fama–MacBeth with a pre-committed gate is the right architecture. It is wired to the weaker of the two standard errors the repository itself implements (F3), and the surviving coefficients carry uncorrected selection bias.
Signal & gates	STRONG	STRONG	WEAK	UNKNOWN	Stage 8's funnel accounting is genuinely good. Stage 6 refuses 1.55 of the model's top 8 on average, on a criterion the model was not fitted under (F5).
Portfolio & risk	ADEQUATE	WEAK	FAILING	FAILING	Sizing and cost modelling are careful and correct. The simulator rebalances quarterly; the engine runs daily. Drawdown is measured on quarterly returns and understates by 3–4×. No benchmark exists anywhere.
Validation	ADEQUATE	FAILING	FAILING	FAILING	Purge and embargo are correct. The Deflated Sharpe is computed on 639 pooled observations where 71 distinct dates exist, with a variance term that is both wrong and contradicts its own docstring (F1).
Operations	STRONG	N/A	FAILING	N/A	Locking, job slots, alerting, archiving and the trial registry are well built. Six config versions in 69 run dates; the forward test is already invalid; 63 of 69 live runs carry a survivorship flag.

P0 — correctness and validity threats

Five defects that change what the system computes or what its numbers mean. Each was found by reading the code and confirmed by running it.

F1 The Deflated Sharpe Ratio cannot fail

CRITICAL

`CpcvResult.deflated()` passes the *pooled* excess vector to `deflated_sharpe_ratio`. That vector has one entry per (split, test-date) pair: 639 entries covering 71 distinct panel dates — each date counted about nine times, and about 28 times over against the number of independent 63-session windows. The PSR's $\sqrt{n-1}$ term therefore runs on a sample size inflated roughly five-fold.

Compounding it, `trial_sharpes` is never supplied by any caller, so the fallback fires:

```
sr_var = 1.0 / max(n - 1, 1)    # docstring: "a conservative unit variance,
                                # which makes the benchmark harder to clear"
```

At $n = 639$ that is 1.57×10^{-3} , not 1.0. The comment claims the fallback is conservative; the code is anti-conservative by a factor of 639. And the quantity itself is wrong in kind: Bailey & López de Prado's $\text{Var}[\text{SR}]$ is the *cross-sectional dispersion of Sharpe ratios across the trials tried*, not the sampling variance of one estimate. The right input — the nine woven path Sharpes — sits unused in the same object.

trials charged	benchmark SR	DSR	verdict	
1	+0.0000	1.0000	PASS	
44	+0.0882	1.0000	PASS	<- what the README quotes
81	+0.0972	1.0000	PASS	<- what the registry now holds
5,000	+0.1460	0.9976	PASS	
100,000	+0.1738	0.9826	PASS	
10,000,000	+0.2100	0.8808	FAIL	

Under any honest accounting of the same data the result reverses:

```
as shipped (n = 639 pooled pairs, sr_var = 1/(n-1))    DSR 1.0000  PASS
n = 71 distinct panel dates                          DSR 0.4649  fail
n = 23 independent 63-session windows                DSR 0.1477  fail
shipped n, sr_var = 1.0 as the docstring claims      DSR 0.0000  fail
independent windows + sr_var from the path Sharpes   DSR 0.3130  fail
```

Why the existing tests miss it. Both DSR tests pass `trial_sharpes` explicitly. They exercise a branch production never takes. **Contamination:** every "DSR pass" in the README and in `BASELINE_V1.json`.

Bitter irony: the commit history shows real, careful work replacing a hand-typed trial count with an append-only registry — a genuine improvement, fed into a statistic that could not respond to it.

F2 Every live score is computed from features four trading sessions stale**CRITICAL**

Both live scoring paths — `fit_predict`'s inference branch and `today_features` — call:

```
live_hist = hist[live_cols].tail(MIN_LOOKBACK + 5)          # 279 rows
live      = build_panel(live_hist, horizon=1, step=21)

# build_panel:  for i in range(MIN_LOOKBACK, len(dates) - horizon, step)
#              =  range(274, 278, 21)  ->  [274]
# dates[274] is four rows before dates[278]
```

With `horizon=1` the loop's upper bound is `len-2`, so it is *structurally impossible* for this construction to reach the decision date at any horizon. The +5 buffer and the 21-session step land it exactly four sessions back. Confirmed live against the store:

```
as_of (decision date)          2026-08-25
LIVE FEATURE DATE ACTUALLY USED 2026-08-19
sessions in between            4  [08-20, 08-21, 08-24, 08-25]
```

Measured cost, running the live coefficients over 88 panel dates at feature lags 0, 1, 2, 4 and 8:

lag	rank IC	ICIR	adj t	top-dec excess	hit	top-8 agreement	
0	+0.0751	+0.473	+2.59	+0.98%	73%	100%	
1	+0.0753	+0.478	+2.61	+0.89%	73%	83%	
2	+0.0742	+0.470	+2.57	+0.95%	74%	74%	
4	+0.0730	+0.465	+2.53	+0.83%	74%	64%	<- shipped
8	+0.0746	+0.479	+2.61	+1.17%	70%	52%	

The statistical cost is inside the noise band — lag 8 scores better on excess, which is the tell. The *operational* cost is not: **only 64% of the top eight names agree with the top eight the same model produces on the decision date.** Roughly three of the eight names on any card are last Tuesday's names, priced at today's close, with today's ATR stop and today's entry zone attached to a thesis computed four sessions earlier.

Fix. Compute the decision-date row directly via `_features_at(close, turnover, len(dates)-1, bench)` rather than routing a single date through the panel builder's stride loop. **Regression test:** assert `today_features(...)[\"date\"].max() == as_of` — no such assertion exists, and I verified that *fixing* the bug breaks no test.

F3 The significance gate uses the correction the repository itself argues is insufficient CRITICAL

`validation/significance.py` is one of the best pieces of work in this codebase. It derives the variance inflation the 63/21 sampling scheme induces analytically, and documents that estimating it instead recovers only 1.74 where the arithmetic gives 3.00 at small n . That module is used for every *reported* figure.

It is not used by the estimator that decides which themes are traded. `famamacbeth.newey_west_se` is a plain Bartlett sum at two lags. Measured on the real 83-date slope series:

analytic VIF for $h=63$, $s=21$, $n=83$: 2.9679 (sqrt 1.7228)

theme	lambda	naive t	NW2 t	estVIF	analytic t	gate@ t >=2
mom_f	+0.0764	+5.60	+4.12	1.85	+3.27	keep -> keep
delivery_f	+0.0474	+5.73	+4.77	1.44	+3.34	keep -> keep
lottery_f	-0.0455	-2.95	-2.26	1.71	-1.72	keep -> KILL
skew_f	-0.0190	-2.51	-1.84	1.86	-1.46	kill -> KILL
reversal_f	+0.0182	+1.73	+1.69	1.06	+1.01	kill -> KILL
beta_f	+0.0087	+0.73	+0.52	1.99	+0.43	kill -> KILL
drawdown_f	+0.0090	+0.68	+0.51	1.78	+0.40	kill -> KILL

The estimated inflation lands at 1.44–1.99 against an arithmetic 2.97 — exactly the failure the module documents. Under the repository's own preferred correction `lottery_f` fails the pre-committed floor and the model drops to two themes.

And the correction improves the strategy. Out of sample over 45 purged splits, `mom + delivery` is the best configuration tested on every measure: +1.57% top-decile excess against +1.25% for the shipped three, +1.61% net book return against +0.55%, Sharpe +0.58 against +0.22. This is the rare case where the statistically correct fix is also the profitable one.

F4 The universe price floor is applied to back-adjusted prices**CRITICAL**

The store back-adjusts `open/high/low/close` on every read — correct for returns. But both universe screens then apply `min_price_inr = 20` to the adjusted series:

```
liquidity_mask:      ok = (adtv >= floor) & (close >= min_price_inr) & ...
resolve_liquidity_pit: keep = [s for s in eligible if last.get(s) >= min_price_inr]
```

A stock that traded at ₹200 in 2019 and later did a 1:20 split now reads ₹10 on that 2019 date and fails a floor it comfortably cleared at the time. Measured against the raw bhavcopy close:

```
(date,symbol) cells with both prices          3,848,322
cells where adjustment moved the price        434,710  (11.30%)
```

price floor of Rs 20 on the ADJUSTED series:

```
cells excluded that would NOT be on the raw series    58,456
cells kept that WOULD be excluded on the raw series    0
distinct symbols affected                             165
```

worst affected (sessions wrongly excluded):

```
TTL 1759  CUPID 1696  IRB 1551  SALASAR 1515  NDL 1443
RAMASTEEL 1361  WEBELSOLAR 1332  PCJEWELLER 1314
```

Membership in a past universe depends on a corporate action that had not yet been announced. The bias runs one way only, and it removes names that later split — disproportionately past winners — so the direction is conservative for measured momentum. It is still look-ahead, it makes the historical universe non-reconstructable in real time, and it means the live screen and the backtest screen apply different tests to the same name.

Fix. The store already carries `adj_factor`. Apply the floor to `close / cumulative_adj_factor`, or read one unadjusted close column for the screen. **Same class:** `panels["adtv"] = (close * volume).rolling(21).mean()` in the portfolio harness multiplies adjusted close by raw volume, understating historical turnover for every name that later split.

F5 The model now ranks a population it cannot buy from — a regression introduced at HEAD CRITICAL

`exits.tradeable_at_entry` refuses a name already below $MA(50) - 1.5 \times ATR$: it satisfies its own first exit condition at the moment it is opened. Stage 6 enforces this live (`require_above_invalidation: true`). Under the barrier label the *training panel* enforced it too, through `resolve_exits` , so the two populations matched.

The last commit set `triple_barrier: false` . `build_panel` now takes the plain-forward-return branch, `resolve_exits` is never called, and the admissibility mask disappears from training — visible in the model blob as `n_train` jumping 35,060 → 43,555 (+24%). Stage 6 still refuses. Measured over 86 panel dates:

share of the SCORED universe below its invalidation level:

mean 23.3% median 17.6% max 88.0%

of the model's top 8 on each date, Stage 6 refuses 1.55 names on average

median 1, max 6

dates where at least one of the top 8 is refused: 72%

dates where half or more of the top 8 is refused: 12%

So the shipped shortlist is not the model's top eight; it is the top eight of a subset the model was not fitted on, and roughly one name in five on the card arrived by promotion rather than by rank.

Fix — pick one and state it. Either apply `tradeable_at_entry` as a panel mask when the barrier label is off, or drop `require_above_invalidation` and let Stage 7 place a stop that is already breached. The current state is the only combination that is definitely wrong.

False-confidence register

Mechanisms that make ProSignal look better, safer or more established than the evidence supports. This is the section the mission called most important, and it is the one where the system is weakest.

C1 A t-statistic is refused and a Deflated Sharpe is printed from the same arithmetic

CRITICAL

`research cpcv` prints, verbatim: *"No t-statistic is quoted... neither is a sample of independent experiments and a t from either is inflated."* That refusal is exactly right. It then prints a Deflated Sharpe computed on that same pooled vector. The DSR is a t-statistic wearing a probability's clothes — $\Phi((SR - SR_0) \sqrt{(n-1)/\sigma})$. The engine declines to quote the inflated number and quotes its monotone transform instead.

C2 Reported drawdown understates the real one by three to four times

CRITICAL

`PortfolioResult.metrics` computes `(1+ret).cumprod()` over 63-session *period* returns, inside a single CPCV split. It cannot see an intra-quarter fall, and the splits are short. The README's "max drawdown median -6.9%" is a quarterly-frequency drawdown over a partial window. Running the same out-of-sample rankings as one continuous book over the whole selection period gives **-20.4% to -27.4%**.

C3 The recorded outcome record is negative and appears nowhere

CRITICAL

The engine keeps a permanent outcome ledger. Under the current exit model:

```
CURRENT exit model (target-t2-v4)          n = 118
  exit_reason book_exit 105 (89%) stop 12 stop_gap 1 target 0
  net      mean -0.51% median -0.57% sd 5.04% win 43%
  gross   mean -0.13% cost drag 0.37%
  held    median 3 sessions (against a 63-session model horizon)
  config_versions spanned: 8
```

```
SUPERSEDED rows          n = 716
  net      mean +5.02% median +10.75% win 59%
```

The exit-model correction was the right call — the +5.02% figure was computed over phantom sessions the book never held. But the corrected number, **-0.51% mean net across 118 trades**, is not stated in the README, which still leads with selection-period ranking figures. The honest caveat cuts both ways: those 118 trades span eight config versions and a universe the engine no longer uses, so they are not evidence *against* the shipped system either. They are evidence that no live record of the shipped system exists.

C4 The live operating history is on a different universe from the validated one**CRITICAL**

From the 2026 run ledger — 1,679 rows across 69 recorded dates:

date	config	univ	elig	scored	buys	survivorship_risk
2026-07-28	@4796c401fe1ffa54	200	10	10	0	True
2026-08-06	@308bcf7214e989d6	200	16	16	0	True
2026-08-14	@3c3c1f3f17a13929	200	18	18	0	True
2026-08-17	@3c3c1f3f17a13929	200	145	145	2	False
2026-08-18	@3ecb5458f5632551	750	640	640	8	False
2026-08-21	@39e9687c57bd12bd	750	617	617	8	False
2026-08-25	@9776e5d6b3a3dd3e	750	453	453	8	False

universe_considered values seen: {200: 65 dates, 750: 4 dates}
 survivorship_risk flag raised on 63 of 69 dates
 53 of 69 runs produced NO TRADE (21 of those blocked by regime)

Until 2026-08-18 the engine ran on a survivorship-flagged NIFTY 200 snapshot and scored ten to eighteen names. **The shipped 750-name point-in-time configuration has run on four dates and has produced zero resolved outcomes.** Runs were recorded on 69 of roughly 160 trading sessions — 43%, below the forward test's own 60% coverage invalidation threshold.

C5 The forward test is invalid before its first observation — for at least the third time**CRITICAL**

registration on disk	started_on 2026-08-27	config baseline-v1@a30a8d4847080ddc
	fingerprint 04a39fb964483900	
README says	registered 2026-08-23	config baseline-v1@39e9687c57bd12bd
	fingerprint ce83da4347d90323	
HEAD commit note says	"the config hash moved to baseline-v1@5ffc70e01ee1ba1d"	
config at HEAD computes	baseline-v1@127d8a314ec49aa2	

Four different config versions for one registration. The registration's own *first* invalidation condition is "config_version changes during the window". The 2026 ledger shows six distinct config versions across 69 run dates. The forward test is the number-one item on the project's own research roadmap and the only remaining source of independent observations, and the configuration will not hold still long enough to start it.

C6 The README's headline numbers are stale and mutually inconsistent**HIGH**

Not a research defect, but the document is the system's primary interface and it currently misleads:

CLAIM	ELSEWHERE IN THE SAME FILE / IN THE CODE
Pooled rank IC +0.0682 , 24 trials (exec summary)	+0.0288 , 44 trials (\$CPCV); 0.062 (\$Statistical validation). I measure +0.0567 .
Holdout figures "withdrawn and NOT replaced"	\$Holdout still prints +4.35% / IC 0.1069 / t 3.13, and "What has been established" lists it as reasonably supported .
PBO 44.3%	PBO 35% in the section immediately above.
Value coverage 10–12% , 192 of 750 names	65–73% within-date, 758 names, after the ingest. Live blob: 0.3765 date-span.
Trial count 44 ; "cumulative_trials_logged: 0 — never accumulated"	Registry holds 61 distinct trials; config carries 20 carried → 81 charged. HEAD's own commit says 81.
Family coefficients: 5 families, mom +0.0250 ...	Live model has 7 families; mom is +0.0764 (λ) / +0.0298 (stored scale).
\$"The label is the trade, not the horizon return"	<code>triple_barrier: false</code> . HEAD removed it.
Stage 5: eight checks listed as active	Five are <code>enabled: false</code> at HEAD.
"All ranks are taken within sector"	58% of rows carry a sector; ~46% of names per date are ranked within one.
Model abstains below 376 sessions	<code>MIN_LOOKBACK + H + 60 = 397</code> since <code>prox_52w</code> went to 274.

C7 417 parameters, none validated**HIGH**

TOTAL PARAMS 417

```

UNVALIDATED    127    (explicit)
short form     235    (status silently becomes UNVALIDATED)
OPERATIONAL     29
STRUCTURAL     19
STATUTORY       7
VALIDATED       0

```

The config's honesty discipline is genuinely good — it forces every knob to declare its own evidential status, and the loader refuses VALIDATED without a ledger trial id. The result of applying it is that **362 of 417 parameters are unvalidated hypotheses**, and the engine says so on every card. That is admirable, and it is also an enormous unpriced degree-of-freedom surface. The DSR exists to charge for it and cannot (F1).

The economic question the system never asks

ProSignal measures top-decile excess over the cross-sectional mean — a long-short quantity — and trades a long-only eight-name book. It has never compared that book to buying the market. There is no benchmark anywhere in `portfolio_sim`, `harness` or `backtest`.

I wrote a second, independent continuous-book simulator to cross-check the repository's own: positions opened at a rebalance, carried across rebalances while inside the 8/16 band, closed by the band or by the engine's own stop, 3R target or invalidation level, with the shipped cost model and shipped sizing. Rankings are the same purged out-of-sample CPCV predictions used everywhere above.

SELECTION PERIOD, 2019-02-18 → 2025-02-03 (6.0 YEARS)				
STRATEGY OR BENCHMARK	TOTAL	CAGR	SHARPE	MAX DD
Nifty Alpha 50 INVESTABLE	—	+27.2%	+0.86	−38.4%
Equal-weight eligible universe the engine's own benchmark for beta and residual momentum	+251.6%	+23.5%	+1.16	−42.3%
Nifty Midcap 150 INVESTABLE	—	+22.8%	+0.86	−38.7%
Nifty200 Momentum 30 INVESTABLE, ~30 BPS a listed index expressing the same thesis on the same universe	+144.8%	+16.2%	+0.84	−28.4%
Nifty 200	+134.6%	+15.4%	+0.79	−38.2%
ProSignal book , 21-session rebalance	+21.3%	+3.4%	+0.29	−27.4%
ProSignal book , 63-session rebalance SHIPPED CADENCE	—	−0.8%	−0.01	−20.4%

A researcher can spend a year proving a ranking carries information, and lose the whole argument to a thirty-basis-point index fund that expresses the same idea without a stop.

But the crash property is real, and it is the strongest book-level result in the system

Splitting by regime shows what the stop actually buys:

BOOK VS BENCHMARKS BY REGIME				
PERIOD	PROSIGNAL BOOK	EW UNIVERSE	NIFTY 200	MOMENTUM 30
2019-02 → 2020-01 · pre-covid grind	−4.5%	+12.4%	+12.3%	—
2020-01 → 2020-03 · COVID crash	−4.6%	−34.7%	−28.6%	—
2020-03 → 2021-12 · rebound	+76.2%	+175.3%	+107.9%	—
2022 · drawdown	−15.0%	+2.3%	+3.6%	−6.4%
2023 → 2025-02 · bull	−11.0%	+70.2%	+35.7%	+52.5%

In February–March 2020 the book lost 4.6% while the universe lost 34.7%. That is a thirty-point protection, and it is not luck: 32% of positions close on the stop and 39% on thesis invalidation, both of which fire early in a market-wide fall. The engine has built a very effective crash overlay.

It loses in every other regime, including –11.0% during the 2023–25 bull market while the momentum index made +52.5%. The mechanism is the same one: 65% of positions close on a loss-shaped exit and only 18% reach the 3R target, and the book sits partly in cash between rebalances. In a trending market that is fatal.

THE HONEST REFRAMING

ProSignal is not an alpha engine that happens to have a stop. It is a **drawdown-managed momentum sleeve**, and it should be argued for on that basis: a Sharpe of +0.29 with a –27% worst drawdown against a market Sharpe of +0.79–1.16 with –38% to –42%. That is a real product with a real audience, and it is a different product from the one the README describes.

The unit economics

The cost model is careful and I reproduced its headline: at ₹1.25L into ₹20 cr ADTV the round trip is **87 bps** (STT 20 bps both legs, impact 50, spread 10, stamp/exchange/SEBI/GST/DP/brokerage 7). The engine's own top-decile excess is +0.87% to +1.14% per 63-session period, or roughly 3.5–4.5% a year gross. Turnover measures ~90% per rebalance even with the 8/16 band. **Modelled gross edge and modelled cost are the same order of magnitude.** And the impact coefficient carries status: UNVALIDATED with the note "UNCALIBRATED. Must be fitted against your own realised slippage before any net-of-cost claim is made."

Ablation and the compounded evidence ledger

45 purged CPCV splits over the selection period, out of sample, with the real cost model and real sizing. Excess t is computed on 71 *distinct* dates with the analytic overlap inflation — not on the pooled split $\times$ date vector.

WHAT EACH THEME ADDS · RANKING LEVEL AND BOOK LEVEL							
CONFIGURATION	RANK IC	EXCESS	T (ADJ)	NET/PERIOD	DEPLOYED	RET/DEPLOYED	SHARPE
mom alone	+0.0617	+2.06%	+2.31	+1.72%	71%	+2.44%	+0.55
mom + delivery ANALYTIC GATE	+0.0581	+1.57%	+1.92	+1.61%	74%	+2.17%	+0.58
mom + delivery + lottery SHIPPED GATE	+0.0571	+1.25%	+1.42	+0.55%	78%	+0.70%	+0.22
ALL 7 themes SHIPPED	+0.0548	+1.14%	+1.42	+0.35%	78%	+0.45%	+0.12
drop lottery	+0.0461	+0.62%	+0.86	+0.74%	79%	+0.94%	+0.27
drop delivery	+0.0431	+0.19%	+0.23	+0.89%	84%	+1.06%	+0.33
drop mom	+0.0525	+0.14%	+0.16	−0.50%	86%	−0.59%	−0.26

THE COMPOUNDED LEDGER, READ PLAINLY

- **Baseline** → **+mom**. Momentum is the whole engine. Removing it takes the book to −0.50% per period at Sharpe −0.26 and the excess to statistical zero.
- **+delivery**. Adds ranking information (excess +0.19% → +1.14% when present; IC +0.0431 → +0.0548) and holds the best ICIR of any family, +0.509. At the book level it is roughly neutral. Keep it: it is the only genuinely independent bet in the system (ρ with mom = −0.028).
- **+lottery**. **Costs 1.06 percentage points of return per period and 0.36 of Sharpe**. The single most damaging inclusion, and the one that should not be there on the repository's own inference standard.
- **+skew, +beta, +drawdown, +reversal**. All zeroed by the gate today. The gate is doing its job.
- **Complexity verdict**. Seven families, twelve computed factors, eleven more built and dropped, ~1,400 lines of estimator and orchestration — delivering a model that is beaten by `rank(mom_f)`.

Not one configuration clears the project's own bar

The config declares `validation.significance.t_stat_bar: 3.0`, citing Harvey, Liu & Zhu (2016), and states in the README that "the bar is stated before the test." Applying the repository's own analytic overlap correction to 71 distinct out-of-sample dates:

```
best configuration (mom alone)    adj t +2.31 < 3.0
mom + delivery                   adj t +1.92 < 3.0
shipped 7-theme model            adj t +1.42 < 3.0
best single factor IC (prox_52w) adj t +2.44 < 3.0
best family IC (delivery_f)      adj t +2.63 < 3.0
```

The bar was set in advance, honestly, and nothing in the system clears it. The README does not say this.

Attribution: there is no residual alpha

Regressing the strategy's out-of-sample top-decile excess on long-short factors built from the engine's *own* family definitions, on the same universe and dates — a deliberately hostile test:

FACTOR SET	R ²	ALPHA/PERIOD	NAIVE T	ADJ T	SURVIVES
Market only	0.265	+1.88%	+4.36	+2.54	YES
Momentum only	0.329	+0.06%	+0.14	+0.08	NO
Momentum + delivery	0.519	-0.44%	-1.19	-0.69	NO
All seven families	0.645	-1.29%	-2.98	-1.73	NO

Momentum alone explains the entire top-decile excess. Only two loadings clear $|t| \geq 2$ after correction: `delivery_f` at +2.67 and `mom_f` at +2.13 — the same two the analytic gate would price. Everything is consistent; there is simply no alpha left over. $n = 66$ overlapping windows $\rightarrow$ ~ 22 independent observations against 8 parameters, so this is directional, not conclusive.

Factor registry

Standalone rank IC over the 83-date shipped panel against the 63-session forward return, with the analytic overlap correction applied to the t-statistic.

The README quotes naive t-statistics for these without saying so.

TWELVE SCORED FACTORS + TWO UNSCORED DIAGNOSTICS							
FACTOR	IC	ICIR	NAIVE T	ADJ T	HIT	FAMILY	VERDICT
prox_52w close 21-back / 252-session high ending 21-back – 1	+0.0676	+0.458	+4.20	+2.44	70%	mom	KEEP
max_dd_120	+0.0574	+0.345	+3.16	+1.83	61%	drawdown	SUBSUMED BY MOM
resid_mom	+0.0503	+0.406	+3.72	+2.16	70%	mom	KEEP
mom_6_1	+0.0490	+0.386	+3.53	+2.05	71%	mom	KEEP
deliv_pct no published literature exists for this factor	+0.0462	+0.413	+3.67	+2.13	60%	delivery	KEEP
deliv_trend	+0.0251	+0.380	+3.36	+1.95	66%	delivery	KEEP
resid_reversal	+0.0116	+0.126	+1.15	+0.67	52%	reversal	NO SIGNAL
turnover_ratio	–0.0047	–0.051	–0.47	–0.27	45%	—	CORRECTLY UNSCORED
amihud	–0.0089	–0.080	–0.73	–0.43	54%	—	CORRECTLY UNSCORED
idio_skew	–0.0235	–0.281	–2.57	–1.49	47%	skew	ZEROED
beta_120	–0.0270	–0.166	–1.52	–0.89	48%	beta	ZEROED
max5_21	–0.0284	–0.232	–2.12	–1.23	41%	lottery	MARGINAL
downside_vol	–0.0445	–0.317	–2.90	–1.69	41%	lottery	MARGINAL
idio_vol	–0.0538	–0.330	–3.02	–1.75	42%	lottery	MARGINAL

Structure: less concentrated than the README says, but only two bets are live

FAMILY columns, within-date rank correlation, mean over 71 dates

	mom	rev	lot	skew	beta	dd	deliv
mom	1.000	.037	.114	.151	.030	.437	-.028
lottery	.114	.192	1.000	.200	.473	-.469	-.362
beta	.030	-.022	.473	.078	1.000	-.347	-.291
drawdown	.437	.183	-.469	.025	-.347	1.000	.114
delivery	-.028	-.167	-.362	-.148	-.291	.114	1.000

PC1 explains 30.1%; 6 of 7 components for 90% of variance
participation ratio (effective independent bets) 5.21 of 7

MEMBER factors, $|\rho| \geq 0.55$:

mom_6_1	resid_mom	+0.730
downside_vol	idio_vol	+0.673
mom_6_1	prox_52w	+0.647
max_dd_120	prox_52w	+0.627
idio_vol	max5_21	+0.572
prox_52w	resid_mom	+0.560

12 members -> 8 components for 90%; participation ratio 6.26

The README's "17 factors span 11 principal components, participation ratio 8.97, the engine is less diversified than the factor count implies" is stale and, at the level the model actually fits, too pessimistic: the seven families carry 5.21 effective independent bets. The real concentration is elsewhere — only three families carry a coefficient, and two of them are the model.

Delivery is the most interesting thing in this system

I searched for published cross-sectional research on NSE delivery percentage as a return predictor and found none — only broker and screener commentary. The code says the same ("NSE-specific; no standard reference"). That has two consequences the repository draws only one of:

- **Favourable.** McLean & Pontiff's 58% post-publication decay prices arbitrage that follows a paper. No paper exists here, so `delivery` is the one factor in the book with a plausible claim to being uncrowded — and it is the highest-ICIR family (+0.509) and essentially orthogonal to momentum.
- **Unfavourable, and unstated.** `decay.py` applies `HAIRCUT_MCLEAN_PONTIFF = 0.58` uniformly to every theme including this one, and the README's headline "delivery at 258% of expected" compares it against a decay expectation that does not apply to an unpublished factor. It is also unreplicated by anyone: a single researcher, one market, one nine-year window.

Research weakness register

Not bugs. Methodological choices that are defensible in isolation and weak as evidence.

W1 The ranking target explains only a quarter of what the book earns

HIGH

The model is fitted against the 63-session forward return. The book earns whatever `resolve_exits` produces under a 2.5×ATR stop, a 3R target and a 50-session invalidation level. Measured across 35,643 rows on 86 dates:

```
within-date rank correlation, label vs realised book outcome
  mean 0.531   min -0.062   max 0.824   (0.531^2 = 28% of variance)
```

```
exit side distribution of the book's own positions
  invalidation 39.3%   stop 32.1%
  target       17.9%   timeout 10.7%
```

```
top-decile minus bottom-decile, by factor:
      raw label   book outcome
mom_f      +4.36%    +1.70%
delivery_f  +1.77%    +1.26%
drawdown_f  +1.52%    +1.09%
```

Momentum's measured edge on the label is 2.6× what the book captures. Every top-decile excess figure in the repository is a label figure.

The tension is real and unresolved. HEAD removed the barrier label for a correct reason — using a volatility-scaled magnitude as a ranking target puts back the heteroskedasticity the scaling exists to remove. But removing it reopened the gap between the target and the payoff. The honest options are a rank-transformed barrier label (which removes the magnitude problem while keeping the path), or accepting the gap and stating it on every excess figure.

W2 Selection bias in the surviving coefficients is uncorrected

HIGH

`gated_shrink` selects on $|t| \geq 2$ from the same data that estimated λ , then shrinks the survivors. The `tau2_from` fix — estimating τ^2 from the pre-selection set — is correct and well argued, and it fixes the winner's curse *in* τ^2 . It does nothing about the winner's curse *in* λ itself: conditional on $|t| \geq 2$, $E[\lambda]$ is biased away from zero, and the bias is largest exactly at the boundary where `lottery` ($t -2.26$) and `risk` ($t +1.86$ live vs $+2.45$ on rebuild) sit.

W3 Sector-neutral ranking is a mixture of two incomparable normalisations**HIGH**

```
rows carrying a sector label                                58.0%
names ranked WITHIN sector per date    median 46.0% (36.2% - 54.9%)
```

on the last panel date:

```
55% of the universe is within-sector ranked
49% of the top mom_f decile is
```

A within-sector rank of +0.9 and a universe rank of +0.9 are different quantities, and both live in the same column for 54% / 46% of the names. The extremes are drawn disproportionately from the unsectored pool. The README states flatly that "all ranks are taken within sector".

W4 Single-theme arms under a gated estimator are scored on selected splits**MEDIUM**

`configuration_matrix` documents this trap precisely, for PBO. The README's estimator comparison table — the one that concludes "the production ridge did not beat the control it exists to justify" — walks into it, and so did my own first ablation. With one theme, `hierarchical_shrink` drives τ^2 to zero whenever $|t| \leq 1$, so a single-theme arm trades only on splits where it happened to measure. I measured the size of the artefact:

```
beta alone, gate ON (20 of 45 splits fit)    IC -0.105    net +1.43%    SR +0.57
beta alone, gate OFF (all splits)            IC -0.016    net -1.25%    SR -0.56
```

Same factor, same data, opposite conclusion. Any single-theme comparison in this repository is unreliable unless the floor was explicitly set to zero.

W5 Delivery is a constant on five of eighty-three cross-sections**MEDIUM**

The delivery feed starts in 2019 and `deliv_pct` is neutral-when-missing, so `delivery_f` is identically zero for every name on 5 dates (2019-02-18 → 2019-06-25). Those dates enter its mean and its Newey–West standard error as if they were measurements.

	all 83 dates		excluding the 5 dead dates		
<code>delivery_f</code>	+0.0474	t +4.77	+0.0505	t +4.87	
<code>mom_f</code>	+0.0764	t +4.12	+0.0704	t +3.72	
<code>lottery_f</code>	-0.0455	t -2.26	-0.0375	t -1.83	<- also falls below the gate

W6 One definition of a trade, except in the simulator**MEDIUM**

`features/exits.py` exists explicitly to end the situation where "the model was fitted against the first, the backtest measured the second, and the engine traded the third." It has two callers. One of them, `portfolio_sim._hold`, passes `high=None`, so the target is tested on the `close` while the stop is tested on the intraday `low`. Conservative in direction, inconsistent with the label's construction, and inside the module whose whole purpose is that there be one definition.

W7 Uniqueness weights are computed and discarded on the shipped path

LOW

`fit_predict` computes per-row average-uniqueness weights (an $O(n)$ loop per symbol) and hands them to `fit_coefficients`, which forwards them only to the `ridge` branch. The shipped `fama_macbeth` branch ignores them entirely. The config declares this honestly and it is defensible — but the labels module's opening argument about overlap ends in an estimator that does not use the remedy.

W8 Small deviations from the cited constructions

LOW

- `resid_mom` is the cumulative residual with *no intercept in the regression* and *no standardisation by residual dispersion*. Blitz, Huij & Martens (2011) do both. As built it retains the name's own drift and scales with volatility, which pulls it toward the lottery block.
- `resid_reversal` 's standardisation window is up to 756 sessions in training but at most 279 on the live path, because `live_hist` is `tail(MIN_LOOKBACK + 5)`. Cross-sectional ranking absorbs most of this, but train and serve are not computing the same statistic.
- Fundamental factors are ranked universe-wide, not within sector, unlike every price factor. Currently moot — they are dropped — but it is the opposite of the stated rule.

Testing the tests

1,343 pass, 1 fails, 49 skip. The suite is genuinely good at pinning stated properties. I injected sixteen defects that would be serious if they shipped; six survived.

MUTATION PROBE · FULL SUITE, WITH THE PRE-EXISTING RED TEST DESELECTED		
INJECTED DEFECT	SUITE	WHY IT MATTERS
Purge disabled in CPCV	CAUGHT	test_purging_actually_removes_overlap
Embargo disabled	CAUGHT	same test
Features read one bar into the future	CAUGHT	test_a_future_price_cannot_move_a_feature
Newey–West degraded to the naive SE	CAUGHT	—
Significance floor lowered to zero	CAUGHT	—
Shrinkage disabled	CAUGHT	—
Label redefined to a past return	CAUGHT	—
Costs zeroed	CAUGHT	—
Sector-neutral ranking removed	CAUGHT	—
Delivery panel silently neutral-filled	CAUGHT	—
DSR hardcoded to 1.0	CAUGHT	but see below
Live scoring made to use the decision date (i.e. <i>fixing</i> F2)	SURVIVED	Nothing anywhere pins the live feature date. The bug can be fixed or made worse invisibly.
Stop-distance 2%–15% clip removed	SURVIVED	A live risk control that also drives position size, untested.
Liquidity screen window semantics changed	SURVIVED	The screen could silently change and nothing would notice.
Uniqueness weights removed from the fit	SURVIVED	Correct — they are already inert on the shipped path (W7).
Training-horizon truncation removed	SURVIVED	Correct — see Rejected hypotheses.

T1 The two tests that certify the DSR exercise a branch production never takes**CRITICAL**

```
def test_dsr_collapses_as_trial_count_rises():
    few = deflated_sharpe_ratio(r, n_trials=1, trial_sharpes=[0.15])
    many = deflated_sharpe_ratio(r, n_trials=5000, trial_sharpes=list(...))
    assert few.deflated_sr > many.deflated_sr

# grep trial_sharpes across src/ (excluding metrics.py itself): 0 hits.
# The only production call site:
# harness.py:105 return deflated_sharpe_ratio(self.excess, n_trials=n_trials)
```

Both DSR tests supply `trial_sharpes` explicitly. Production never does. The tests prove the DSR responds to the trial count *when the trial distribution is given*; on the branch the engine actually runs it passes at 100,000 trials. This is the cleanest example in the codebase of a test faithfully confirming a specification that is not the one in force.

T2 The suite is red on a clean checkout**MEDIUM**

`test_ready_freshness.py::test_latest_session_is_reported_on_every_branch` asserts `latest_session` is truthy on every branch of `/ready`. On an empty store it is legitimately `None` — and the sibling test three lines below explicitly skips on exactly that condition. `data/curated/` is gitignored, so a fresh clone has an empty store and the suite fails. HEAD's commit message reports "1,384 passed, 0 failed"; that was measured against a populated local store. The same commit fixed nine other tests for precisely this class of problem and introduced a tenth.

WHAT THE SUITE ESTABLISHES, AND WHAT IT DOES NOT

It establishes that the code does what it was written to do, and it does that unusually well — the purge test, the future-price test, the label test and the sector-rank test are real property tests, not implementation echoes. What it does not establish, and cannot: that the specification is right. The gaps cluster in exactly one place — **nothing tests the live scoring path end to end against the store**. F2, F5 and half of W8 all live there.

Missing capability register

The absence of a control is often more consequential than a bug in one that exists. Ranked by what each would change.

1	A benchmark, anywhere. No book-level code path compares the book to an index, an equal-weight universe, or a passive alternative. Every economic conclusion in the repository is stated against zero. This is the finding that reverses the verdict.	CRITICAL
2	A continuous-book simulation at the operated cadence. <code>simulate</code> advances one cohort every 63 sessions; the engine runs every session and re-applies the 8/16 band each time. The recorded outcome ledger shows a median hold of 3 sessions and 89% <code>book_exit</code> . The book that was validated and the book that runs are different books.	CRITICAL
3	Path-level drawdown. Drawdown computed on 63-session period returns within a split. The real number is 3–4× larger.	CRITICAL
4	A regression test on the live scoring date. One assertion — <code>today_features(...)[“date”].max() == as_of</code> — would have caught F2 the day it was written.	HIGH
5	A model fingerprint in the ledger. The README lists this as roadmap item 2 and it is still open. The config hash does not move when the store grows, so the same hash produces materially different coefficients — and the forward test's integrity check reads only the config hash.	HIGH
6	Capacity and slippage calibration. <code>max_participation_of_adtv</code> is flagged in the config as “THE single most important parameter in this system” and the impact coefficient beside it says “UNCALIBRATED”. Neither has been touched.	HIGH
7	A regime split of the book. The README analyses momentum-crash exposure in the <i>ranking</i> . Nobody has split the <i>book</i> by regime — where the single best result in the system lives (−4.6% vs −34.7% in the COVID crash).	HIGH
8	Calibration of the score. Correctly and repeatedly disclaimed. Still absent, and still the thing a user most wants.	ACKNOWLEDGED
9	Turnover accounting at the operated frequency. Turnover measures ~90% per rebalance in the quarterly simulation. Nobody has measured it at daily re-ranking, where the ledger says the median name survives one session on the screen.	MEDIUM

What is genuinely strong

Adversarial review is not a licence to be indiscriminate. Several things here are better than what I would expect from a professional research platform, and one is better than most of them.

S1 The system reproduces exactly, from its own store, on the first attempt

EXCEPTIONAL

Seven λ , seven t-statistics, `n_train` and `n_dates`, all matching the deployed model blob to four decimal places, reached by calling the engine's own functions on the engine's own data. Most research platforms I have seen cannot do this at all. It is the direct product of the `prepare_features` / `fit_coefficients` / `build_research_panel` extractions, each of which exists because a drift between research and production was caught and closed.

S2 The book simulator does not manufacture edge from noise

STRONG

I ran the null test the repository never ran — five random rankers and two inverted ones through the shipped simulator:

arm	rank IC	net	gross	SR	deploy	hit
shipped 7 themes	+0.0587	+0.35%	+0.85%	+0.12	78%	52%
momentum alone	+0.0617	+1.72%	+2.13%	+0.55	71%	56%
INVERTED momentum	-0.0617	-0.79%	-0.56%	-0.39	50%	32%
INVERTED 7 themes	-0.0587	-0.47%	-0.20%	-0.19	52%	40%
RANDOM ranking #1..#5	~0.000	~+0.03%		+0.01	67%	~44%
(Sharpe range -0.09 .. +0.07)						

Monotone in ranking quality, centred at zero on noise, symmetric under inversion. I went looking for a simulator that flattered the strategy and did not find one. That is a real and load-bearing negative result.

S3 validation/significance.py is excellent work

STRONG

The analytic variance-inflation derivation — $\rho_k = (m-k)/m$ for h-session sums sampled every s, with a small-sample lag weight — is correct, well argued, and better than the estimated alternative at this sample size. The bootstrap gating at $n \geq 30$ is a genuinely contrarian call backed by a simulation the author ran, and it is right. The tragedy is that the module is not wired to the decision it bears on (F3).

S4 The crash property

STRONG

-4.6% against the universe's -34.7% in February–March 2020, with a mechanism that explains it. This is the best-evidenced positive result about the book in the whole system, and nobody has measured it or claimed it.

S5 Operational discipline**STRONG**

- **Refit gate + model archiving.** A proposed refit is reviewed against the live coefficients before it replaces them, the previous blob is archived under a content digest, and the "unloadable previous is a first fit" path is reasoned out rather than crashed through.
- **Label fingerprinting.** Added at HEAD after the exact failure it prevents actually happened. Precisely the right response to a bug.
- **Trial registry.** Append-only, keyed by (command, configuration) so re-runs do not inflate. Correct design; wasted on F1.
- **Store locking.** Advisory flock, exclusive for ingest and shared non-blocking for analysis, with a named holder in the error.
- **Decay monitor sign orientation.** The pre-committed kill rule compared a raw t against zero and would have deleted the two best themes. Caught, fixed, and — importantly — fixed as an evaluation correction with the floor and the breach count untouched.
- **The cost model.** Correct GST base, both-leg STT, square-root impact, buy-side stamp duty. I recomputed it independently and got 87 bps against the documented 86.

S6 The gate is doing real work, and one apparent failure is actually correct**STRONG**

`drawdown_f` has a standalone IC of +0.0574 at a 61% hit rate — second only to momentum — and the estimator prices it at exactly zero (t +0.51). That looks like a failure and is not: `max_dd_120` correlates +0.63 with `prox_52w`, so the multivariate slope is correctly absorbed by momentum. The same is true of `beta_f`, whose univariate IC is -0.027 and whose fitted sign is positive. The estimator is finding what a univariate IC cannot.

Hypotheses I formed and then rejected

The mission asked for the failed hypotheses as well as the confirmed ones.

These are the ones I was wrong about, and one of them I was wrong about twice.

HYPOTHESIS	WHAT I FOUND	VERDICT
The <code> label < 1.0</code> filter censors the right tail materially. It conditions sample membership on the outcome — a name that doubled inside the window is deleted.	It removes 166 of 46,538 eligible labelled rows: 0.36% . Real in principle, immaterial in fact.	REJECTED
<code>train_close = hist.iloc[:len(hist)-H]</code> is the leakage control its comment says it is. Removing it should leak future labels.	It cannot. <code>build_panel</code> already bounds the loop at <code>len(dates)-horizon</code> , so every label ends at or before <code>as_of</code> regardless. The line is a redundant second 63-session buffer that costs three panel dates. The comment — "the leak this model exists to avoid" — is false assurance : it credits the wrong line, and if <code>build_panel</code> 's bound were ever "optimised" the comment would tell the next reader they were safe.	REJECTED → NEW FINDING
The book simulator turns negative-IC signals into positive Sharpe, so book-level comparisons are uninformative. <code>beta alone</code> read IC -0.105 with net $+1.43\%$.	Wrong twice over. The null test (S2) shows the simulator is monotone in ranking quality. And the <code>beta alone</code> figure was <i>my own</i> gate-selection artefact — with the floor at zero it reads IC -0.016 and net -1.25% . I walked into the trap <code>configuration_matrix</code> documents.	REJECTED – MY ERROR
CPCV path weaving is broken: path id assigned by "k-th time this date was tested" produces arbitrary groupings rather than coherent backtests.	It matches López de Prado's construction exactly. The paths are legitimately woven. The problem is elsewhere — in what is then done with them.	REJECTED
Non-EQ series (bonds, G-secs, debentures) leak into the universe and the benchmark. The raw parquet carries them.	<code>read_prices</code> filters on <code>equity_series</code> at the store boundary, with a docstring explaining the NTPC debenture case that motivated it. Correctly handled.	REJECTED
Retroactive corporate-action adjustment is look-ahead in the features.	For returns it is not — the adjusted series is what a holder experienced, and both sides of every ratio are adjusted consistently. It is look-ahead for level tests, which is F4.	PARTLY
The repository's book-level numbers are inflated.	My independent simulator lands in the same low range as <code>portfolio_sim</code> . The book numbers are not flattered; they are simply never compared to anything.	REJECTED

Highest-leverage work

Ranked by expected research value $\times$ confidence $\div$ implementation cost. Not a list of a hundred improvements — six things, in order.

1	Add a benchmark and re-state every economic claim against it. One column in <code>phase_summary</code> and <code>PortfolioResult.metrics</code>: the equal-weight eligible universe over the same dates, and the Nifty200 Momentum 30 where it exists. Cost: a day. Value: it decides whether the project should continue in its present form. Confidence: total — the data is already in the store.	P0
2	Repair the Deflated Sharpe, then re-run every claim that used it. Three changes: pass <code>result.path_sharpes</code> as <code>trial_sharpes</code>; compute the DSR on per-date means, not pooled split$\times$date pairs; and fix the docstring or the fallback so they agree. Cost: an hour. Value: the project's central defence against selection bias currently cannot fail, and every "DSR pass" on record is void. Confidence: total.	P0
3	Fix the four-session feature staleness, and pin it with a test. Compute the decision-date row directly rather than through the panel builder's stride loop. Cost: an hour. Value: the statistical gain is small, but three of eight names on every card are currently wrong, and the card's price, stop and entry zone are attached to a four-day-old thesis. Confidence: total — verified live.	P0
4	Wire the analytic VIF into the significance gate, and accept the two-theme model. Cost: one line plus a re-fit. Value: the correction is the repository's own argument, and it also delivers the best configuration measured — +1.61% net at Sharpe +0.58 against +0.35% at +0.12. Confidence: high; the ablation is out of sample over 45 purged splits.	P0
5	Simulate the book at the cadence it is actually operated. Continuous positions, daily re-rank, the 8/16 band applied every session. Report path-level drawdown. Cost: a week. Value: the validated book rebalances quarterly and the operated one rebalances daily; the recorded median hold is 3 sessions against a 63-session model. Nobody knows what the live strategy's return distribution is. Confidence: high that the gap is large.	P0
6	Freeze the configuration and start the forward test — properly this time. Hash the training window into the ledger alongside the config version, then do not touch <code>parameters.yaml</code> for eighteen months. Cost: discipline, not code. Value: it is the only remaining source of independent observations, and it has now failed to start three times because the configuration keeps moving under it. Confidence: total — the invalidation is already triggered.	P1

THEN, IN ORDER

- **P1 — Apply the price floor to unadjusted prices** (F4) and rebuild the panel. 165 names and 58,456 cells are currently excluded by corporate actions that had not happened.
- **P1 — Resolve the train/admit population mismatch** (F5). Either mask the panel or drop `require_above_invalidation`; the current combination is the only wrong one.
- **P2 — Rewrite the README against HEAD.** It is the system's primary interface and it currently describes a different engine, with three mutually inconsistent CPCV results and a withdrawn holdout still listed as "reasonably supported".
- **P2 — Delete five families.** `skew`, `beta`, `drawdown`, `reversal` and `lottery` are zeroed or harmful. Keeping them is complexity with a measured negative return.
- **P3 — Calibrate the impact coefficient** against realised fills before any net-of-cost claim is repeated. The config already says this.

- **P4 — Investigate the crash overlay as the product.** The strongest measured result in the system is one nobody set out to find.

Architecture simplification

The engine's complexity budget is badly spent, and the savings are large:

- **Five of seven families** contribute nothing or worse. `FAMILIES`, `THEME_PRIOR_SIGN`, the evidence categories and the card's factor block all shrink with them.
- **Eleven fundamental factors** (`fundamental_factors.py`, `fundamentals.py`, the TTM machinery, the LODR availability derivation, the share-count adjustment) are computed on every run and dropped on every run. They have never scored a name. Gate them behind a feed-availability check rather than running them.
- **The meta-label veto** is built, wired, tested and disabled, with a measured per-date AUC of 0.4996. That is honest, and it is ~600 lines of dead weight in the signal path.
- **The regime multiplier table** is inert by construction: `reachable_multipliers` refuses to apply it because no defensive family is priced, and none can be while value and quality are dropped for coverage. Ten buckets × three multipliers of documented placeholders that reach nothing.
- **Five of twelve Stage 5 checks** are disabled at HEAD as double-counts of priced factors. The right call; now remove them.

A defensible engine here is roughly: the store and its adjustment machinery, two factor families, a Fama–MacBeth fit with an honest standard error, the rank band, the ATR stop, the ledger, and a benchmark. That is perhaps a third of the current signal path, and on this evidence it would perform better.

Would I trust this system's outputs?

FINAL ANSWER

NOT YET

Not "no". The distinction matters, and it is not a courtesy.

Why not "no"

The ranking carries real information. Momentum on this universe measures a top-decile excess of +2.06% per 63-session period at an overlap-corrected t of +2.31 across 45 purged out-of-sample splits, and delivery adds a genuinely independent second bet with the highest ICIR in the system. The leakage controls that matter are real: purge, embargo and the point-in-time panel all survive mutation. The engine reproduces exactly. The cost model is honest. The null test is clean. Almost every claim the repository makes about its own weaknesses is true and is stated before anyone asks.

Why not "yes"

Three reasons, in order of weight.

First: the economic case has never been tested and does not survive being tested. The engine measures a long-short quantity and trades a long-only book, and no code path in it has ever compared that book to anything. When you do, the book returns +3.4% CAGR at Sharpe +0.29 against +16.2% at +0.84 for a listed index expressing the same thesis at thirty basis points, and against +23.5% for the universe it screens. That is not a marginal shortfall. Every hour of the enormous methodological care in this repository has gone into establishing that a ranking works, and none into asking whether the resulting book is worth holding.

Second: the certification is circular. The Deflated Sharpe exists to charge a result for the search that produced it. As implemented it passes at a hundred thousand trials, because it counts 639 duplicated observations as independent and uses a variance term that is both the wrong quantity and the opposite of what its own docstring claims. The trial registry, the PBO, the pre-committed $t \geq 3.0$ bar and the holdout discipline are all real defences — and the number they feed cannot fail. Meanwhile no configuration in the system, including the best one, clears the $t \geq 3.0$ bar the project set for itself before looking.

Third: the thing being validated is not the thing that runs. The validated book rebalances quarterly; the operated one runs every session and the recorded median hold is three sessions. The validated universe is 750 point-in-time names; sixty-five of sixty-nine recorded 2026 runs used a survivorship-flagged 200-name snapshot and scored ten to eighteen names. The live scoring path reads features four sessions stale. The model ranks a population from which Stage 6 refuses one name in five. The forward test is registered against a config hash the engine no longer computes.

The code is impressive. That is not the same as the research being trustworthy, and this system is an unusually clean demonstration of the gap.

What would move it to "yes"

Less than it sounds. Items 1 through 5 of the roadmap are, in total, about two weeks of work, and four of the five are hours rather than days. After them the system would be: a two-factor momentum-and-delivery ranker, measured against the index it must beat, at the cadence it is actually run, with an inference procedure that can return "no". If it still shows

an edge after that, it will be a small one, honestly measured — and that is worth considerably more than the large ones currently on the page.

THE ONE THING I WOULD SAY TO THE AUTHOR

The instinct that runs through this codebase — find the flattering number, prove it is flattering, withdraw it, write down why — is the correct instinct and it is rarer than good code. It has been applied to the ranking with real rigour and it has never once been pointed at the book. The next number to go looking for is the one that says whether any of this beats an index fund, and the data to compute it has been sitting in `curated/indices/` the whole time.

Provenance. Every figure in this dossier was computed in this session by calling ProSignal's own functions against the live curated store at commit a359622: 2,212 sessions, 3,545 symbols, 2017-09-08 to 2026-08-25. The production model reproduces exactly (λ , t , n_{train} , n_{dates}). Panel: 88 dates, 46,364 rows; production-equivalent 83 dates, 43,555 rows. CPCV: 45 splits, 9 woven paths, purge 63, embargo 21. Book simulations use the shipped cost model, the shipped sizing and the shipped exit geometry.

Limits of this audit. The continuous-book simulator in §06 and §14 is my own construction, not the repository's, and is offered as a second opinion; its agreement with `portfolio_sim` on levels is what makes the benchmark gap credible, not the simulator itself. Index comparisons are price returns, not total returns, which understates the benchmarks further. One market, one nine-year window containing an exceptional bull run and one crash. I did not spend the holdout, and I did not re-run any figure the repository has withdrawn.